

Reasons to use Scala for Spark

Eighty eight per cent of Spark users code in Scala for the following reasons:

- 1) Apache Spark is written in Scala and is scalable in JVM. Being proficient in Scala helps you dig into the source code of Spark, so that you can easily access and implement the latter's newest features.
- 2) Spark is implemented in Scala, so it has the maximum features available at the earliest release. The features are ported from Scala to support other languages like Python.
- 3) Scala's interoperability with Java is its biggest advantage, as experienced Java developers can easily grasp the object-oriented concepts quickly. You can also write Java code inside a Scala class.
- 4) Scala is a static typed language. It looks like a dynamic typed language because it uses a sophisticated type inference mechanism. This leads to better performance.
- 5) Scala renders the expressive power of a dynamic programming language without compromising on type safety.
- 6) It is designed for parallelism and concurrency to cater to Big Data applications. Scala has efficient built-in concurrency support and libraries like Akka, which allow you to build a scalable application.
- 7) Scala works well within the MapReduce framework because of its functional nature. Many Scala data frameworks follow similar abstract data types that are consistent with Scala's collection of APIs. Developers just need to learn the basic standard collections, which allow them to easily get acquainted with other libraries.

Installing Scala

Scala can be installed on Windows or Linux based systems. It is mandatory for Java to be installed before Scala. The following steps will install Scala 2.11.7 on Ubuntu 14.04 with Java 7. Type the following commands in a terminal.

1. To install Java, type:

```
$ sudo apt-add-repository ppa:webupd8team/java
$ sudo apt-get update
$ sudo apt-get install oracle-java7-installer
```

2. To install Scala, type:

```
$ cd ~/Downloads
$ wget http://www.scala-lang.org/files/archive/scala-2.11.7.deb
$ sudo dpkg -i scala-2.11.7.deb
$ scala -version
```

Working with Spark RDD using Scala

Resilient Distributed Datasets (RDD) are the basic data types of Spark. They can be created in two ways—from an existing source or an external source.

Creating RDD from an existing source: First, switch to the home directory and load the sparkcontext as sc.

```
$ ./bin/spark-shell
```

Then, create an RDD from existing data stored previously in the driver program. First, we will make an array and then use the `parallelize` method to create the Spark RDD from an iterable already present in the driver program, using the following code:

```
val data = Array(2,4,6,8)
val distData = sc.parallelize(data)
```

To view the content of any RDD, use the `collect` method, as shown below:

```
distData.collect()
```

Creating RDD from an external source: An RDD can be created from external sources which have the Hadoop Input Format such as a shared file system, HDFS, HBase, etc. First, load the desired file using the following syntax:

```
val lines = sc.textFile("text.txt");
```

To display the lines, use the command given below:

```
lines.take(2)
```

Basic transformations and actions

Transformations modify your RDD data from one form to another. Actions will also give you another RDD, but this

Table 1: A comparison between four languages

Metrics	Scala	Java	Python	R
Type	Compiled	Compiled	Interpreted	Interpreted
JVM based	Yes	Yes	No	No
Verbosity	Less	More	Less	Less
Code length	Less	More	Less	Less
Productivity	High	Less	High	High
Scalability	High	High	Less	Less
OOPS support	Yes	Yes	Yes	Yes